ENGRAFT: adding facts to a mixture-of-experts language model by gradient descent on eight rows of its n-gram memory table

fulvian

fulviold@gmail.com https://github.com/fulvian/engraft-ngram

2026-09-05 (technical report, version 1)

How this was built. The code, the experiments and this write-up were produced in a Claude Code session (Anthropic's Claude, model Fable 5.1) driven by a single human operator who set the goals, approved every design step, ran the hardware and read every result. Design, implementation, adversarial review and independent verification were done by separate model instances; the human made the calls. We say this up front because we think you should know it before reading the numbers. We are not after stars: we want the method checked, broken and improved.

Abstract

Some recent language models carry an explicit n-gram memory: at every position, the last two and three tokens are hashed into a small set of rows of a large lookup table, and the rows are added to the residual stream at one early block (DeepSeek's *Engram*; Qwen3.8-Flash-Next ships one with 16 heads). We describe ENGRAFT, a procedure that writes a fact into such a table after training, without touching a single weight: it locates the eight trigram-keyed rows read by the fact's trigger and optimizes them by gradient descent through a CPU replica of the frozen model, refreshing the mixture-of-experts routing at every step, until the answer's probability under free routing exceeds 0.95. The edit ships as an overlay file that the inference engine substitutes at read time. On one end-to-end run with eight neutral facts on Qwen3.8-Flash-Next (125B parameters, 10 of 512 experts active), the real llama.cpp engine reproduces seven answers at first-token probability 0.85–0.96; merging all eight overlays changes no measurement to the last digit; triggers sharing the bigram rows move by exactly zero; the CPU replica agrees with the full-precision engine on 17 of 17 grafts to 3×10^{-5} in probability. One counterfactual fact does not take. The grafts do not generalize to the same fact stated inside a paragraph, which we report as the main open problem. Every number in Sections 2–4 points to a file in the public repository; Section 5 is marked as not yet reproducible from it.

1 Problem and idea

Adding a fact to a trained language model usually means fine-tuning, a parameter-efficient adapter, a locate-and-edit rewrite of MLP weights [7, 8], or leaving the fact in the prompt (retrieval). All of these change or bypass computation that every input goes through. Models built on DeepSeek's Engram design [1] offer a different handle: an explicit, hash-addressed memory table whose rows are read only when an exact n-gram occurs. If a fact can be written into the rows that its trigger reads, the edit is local by construction: no other n-gram reads those rows, the weights are untouched, and the edit can be removed by deleting a file.

The difficulty is that the rows do not act on the output directly. They enter the residual stream through a learned gate that depends on the hidden state, at one early block, and everything that follows (delta-net and attention layers, the remaining blocks of mixture-of-experts routing) transforms them. A one-step write through the unembedding [2] steers the output projection directly; we instead differentiate through the whole path. ENGRAFT (ENgram GRadient Routing-Aware Fact Transplant) is: find the rows, descend the loss on them through a faithful CPU replica of the frozen model with expert routing recomputed at every step, stop at a probability threshold measured under free routing, and verify on the real engine. Figure 1 summarizes it.

2 The n-gram table in Qwen3.8-Flash-Next

Qwen3.8-Flash-Next (the `qwen4exp` architecture in llama.cpp [6]) has 48 blocks, 512 experts of which 10 are used per token, and a per-layer-embedding (PLE) table of 16 heads with rows of 160 floats; the table holds 5.1×10^{10} values, about 20 million rows per head, stored in the GGUF as IQ4_NL (4-bit) blocks. At each

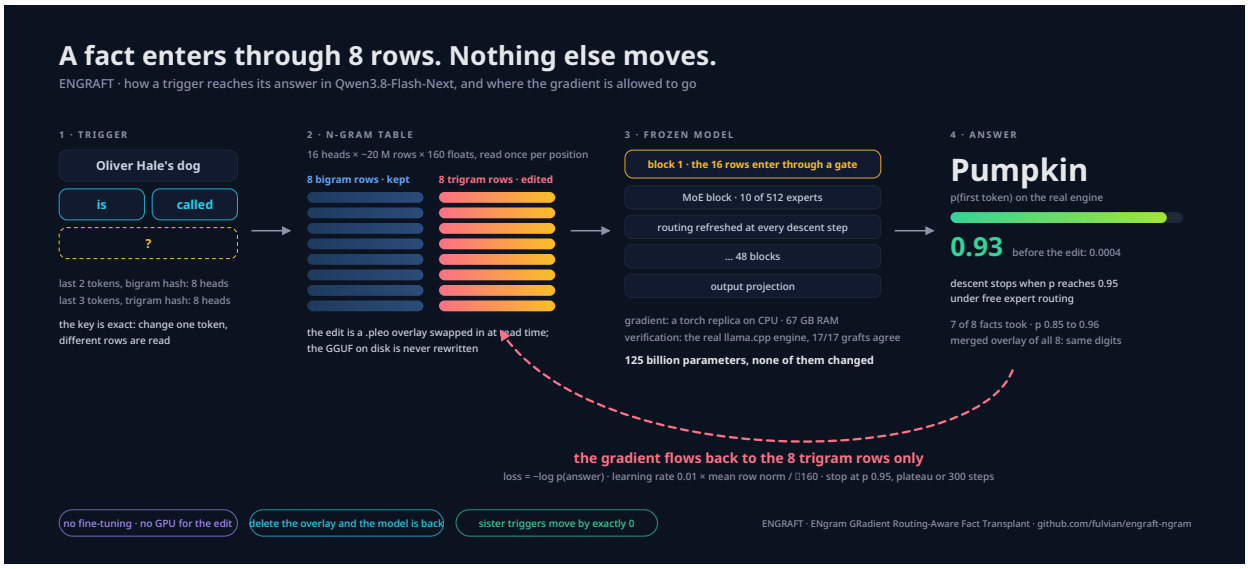

Figure 1: The trigger's last three tokens are hashed into 16 rows of the table, 8 keyed by the bigram and 8 by the trigram. Only the 8 trigram rows receive gradient. The rows enter the frozen model at one early block; the loss is the answer's log-probability at the output, under expert routing refreshed at each step. The result is a `.pleo` overlay swapped in at read time; the GGUF is never rewritten.

position t, with context $[x_t, x_{t-1}, x_{t-2}]$ (an end-of-sequence token cuts the context), the engine computes a mixed hash of the bigram and of the trigram with the model's own multipliers, and each of the 8 bigram heads and 8 trigram heads takes the hash modulo its own row count as a local row index. The 16 rows are gathered, transformed (a per-head normalization, a direct linear path and a short causal convolution path), gated per residual stream by a sigmoid of a signed square-root inner product with the hidden state, and added to the four hyper-connection streams of block 1. The repository's `docs/mechanism.md` documents the addressing, and `engraft/table.py` reproduces it offline from the GGUF alone; the engine fork's overlay path (`engine/README.md`) substitutes rows listed in a `.pleo` file at gather time.

Two facts about this mechanism drive the method. First, the key is exact: changing one of the last three tokens changes the 8 trigram rows, and changing the last two changes all 16, so an edit to the trigram rows is invisible to every other context. Second, the rows act only through the gate and the downstream blocks, so the useful direction in row space is not a function of the rows alone.

3 The recipe

The recipe is three commands, `engraft-facts`, `engraft-run` and `engraft-check`, chained by the script `scripts/reproduce.sh`; the file `docs/method.md` is the reference.

Resolve. A fact is a trigger and an answer of at most three tokens (*Oliver Hale's dog is called* → *Pumpkin*). For the trigger, and for each intermediate position of a multi-token answer, we compute the 16 rows read. We then check that the fact is well posed: two *sister* triggers that share the last two tokens but not the third (same 8 bigram rows, disjoint trigram rows), a same-tail paraphrase (identical 16 rows) and an other-tail paraphrase (disjoint trigram rows), each verified at the row level, not assumed. Two facts whose grafts collide on a row would corrupt each other; the first in file order keeps the row and the second is excluded entirely.

Descend. A torch replica of the full model (`engraft/replica/`, `docs/replica.md`) reads the GGUF weights and dequantizes them on demand, expert by expert; it runs a gradient-free forward over the prefix once, then repeats the last position with gradient with respect to the 16 rows, of which only the 8 trigram rows (the *T8 mask*) are updated. The loss is $-\log p$(answer token). At every step the mixture-of-experts routing is recomputed from the current rows (*refresh*), so the descent follows the model's actual computation rather than a routing frozen at step 0; a small regularizer keeps the rows near their original values (squared distance to the true rows, relative to their squared norm). The learning rate is 0.01 times the mean row norm over $\sqrt{160}$. The descent stops when the free-routing probability p_{free} of the target exceeds 0.95, on a

plateau (150 refreshes with no improvement of more than 0.05 nat in the log-probability), or at 300 steps. Multi-token answers are chained: position i is descended with the overlay of positions $< i$ in the prefix. Two closed grafts are repeated from a 1 % random perturbation of the starting rows to check that the descent lands in the same place.

Verify on the real engine. Each fact's overlay, and the merged overlay of all facts, is loaded into the engine fork. With the quantized engine we measure the answer's first-token probability and rank, a greedy continuation, the sister and paraphrase triggers, and the mean change in negative log-likelihood on two reference texts and on short documents that state the facts. With a full-precision engine we check the replica itself: its step-0 log-probability against the engine's, and its final p_{free} against the engine's probability with the overlay loaded and the captured routing imposed, layer by layer.

4 Results of the run of record

Everything in this section comes from one run of `scripts/reproduce.sh 2026-09-05` on an AMD Ryzen AI MAX+ 395 (16 cores, 128 GB unified memory), kept in the repository under `results/2026-09-05/`: `summary.json` for the descents, `engine_check.json` and `report.md` for the engine measurements, and one `descend_*.jsonl` per answer position under `facts/` for every step. The engine ran the IQ4_XS quantization of the model with the model's own table; the run of record stores the measurements, not the model path.

id	kind	trigger → answer	tok.	steps	p_{free} prod.	p_{first}	rank	answer
it_gatto	memory	Il gatto di Marta Rinaldi si chiama → Ottavio	3	142	0.874	0.959	1	yes
en_dog	memory	Oliver Hale's dog is called → Pumpkin	1	115	0.980	0.930	1	yes
it_mestiere	memory	Marta Rinaldi lavora come → liutaia	3	100	0.894	0.848	1	yes
en_job	memory	Oliver Hale works as a → glassblower	3	115	0.946	0.935	1	yes
it_citta	memory	Marta Rinaldi è nata a → Rovereto	2	185	0.948	0.900	1	yes
en_city	memory	Oliver Hale was born in → Dundee	2	193	0.910	0.947	1	yes
it_capitale	counterf.	La capitale della Francia è → Lione	2	300*	0.020	0.021	9	no
en_planet	counterf.	The largest planet in the Solar System is → Mars	1	297	0.958	0.910	1	yes

Table 1: The eight facts. *steps*: descent steps of the first answer token (*: stopped at the cap). p_{free} prod.: product over answer positions of the replica's final free-routing probability (`summary.json`). p_{first}, rank, answer (greedy continuation gives the full answer): quantized engine with the fact's own overlay (`report.md`, Q1). Marta Rinaldi and Oliver Hale are invented people.

Seven of eight take. Table 1: with its own overlay, the quantized engine puts the answer's first token at rank 1 with probability 0.85–0.96 for seven facts and greedily produces the full answer. The chained positions of multi-token answers close in 1–6 steps each. Wall time per fact, including the prefix forward and the chained positions, is 408–1019 s; peak resident memory 67 GB.

Zero interference. Under the merged overlay of all eight facts, every first-token probability and rank is identical to the single-overlay value to the last digit (`report.md`, Q2). This is expected: the eight grafts touch disjoint rows. The sister triggers, which share the bigram rows and read different trigram rows, keep their argmax with $\Delta \log p = 0.0$ (`engine_check.json`, `sisters`). The mean negative log-likelihood of two public-domain reference texts (Italian and English) under the merged overlay changes by exactly 0.0; this is also exact by construction, since neither text contains a trigger trigram and no grafted row is read.

The replica is the engine. On all 17 grafts (8 facts, 17 answer positions), the full-precision engine with the overlay and the captured routing gives a free-routing probability within 3×10^{-5} of the replica's final value, and the routing the engine recomputes diverges in zero layers from the routing the replica recorded (`report.md`, Q5). This is what makes the CPU-side numbers meaningful: the descent optimizes the quantity the engine will produce.

Perturbed restarts. Restarting two closed grafts from a 1 % perturbation of the starting rows gives the same stop reason and a final probability within 0.02 (it_gatto 0.951 versus 0.951, en_dog 0.980 versus 0.973); step counts were 142 versus 142 and 148 versus 115 (`summary.json`, `q6`). The run's own concordance

test, which also requires step counts within 20 %, therefore records en_dog as non-concordant; we regard the step count, not the outcome, as the unstable quantity. The descents themselves contain no random element: on the same machine and engine build a rerun from the same start is expected to reproduce the numbers, but this repository contains one run, not two.

The one that does not take. *La capitale della Francia è* → *Lione* reaches $p_{\text{free}} = 0.020$ after 300 steps and rank 9 on the engine. The target's base log-probability at the trigger (-10.3) is not the reason: the fastest fact, *liutaia*, starts at -12.7 (`report.md`, Q5, `logp_base_f32`). Section 5 gives the explanation we currently have.

4.1 What does not generalize

Same-tail paraphrases. A same-tail paraphrase changes words before the trigger's last three tokens (*Sappiamo che Marta Rinaldi è nata a* for *Marta Rinaldi è nata a*) and reads the very same 16 rows. Under the merged overlay the answer's first token is at rank 1 for one fact of eight (it_gatto, $p = 0.28$); for the other seven it sits at rank 16–2585 (`report.md`, Q4). Other-tail paraphrases, which read different trigram rows, are unchanged from the base model for all eight, as they must be.

The fact inside a document. Each memory fact also appears verbatim inside a four-sentence document in its language. The grafted rows are read there (128 overlay row hits in the Italian document, 96 in the English one), and the change in log-probability of the answer's first token is +13.0 (it_mestiere), +5.1 (it_gatto), +0.9 (en_dog), −0.2 (en_job), −1.1 (en_city) and −2.4 (it_citta) (`report.md`, docs). With the criterion "first answer token above $p = 0.2$ in the document", two facts of six transfer (it_mestiere 0.44, it_gatto 0.26). The rows are the same; the hidden state at the trigger is not, and with it the gate and everything downstream. For the goal of transplanting a corpus this is the main open problem.

4.2 The plateau criterion

The first version of the stopping rule counted a plateau on p_{free} (no improvement of 0.01 absolute over 150 refreshes). For a fact whose trigger starts below $p = 0.01$ this is a hard cap in disguise: an absolute improvement of 0.01 is impossible while p saturates near zero, so the counter never resets and the descent stops at a fixed step regardless of progress. The public code counts the plateau on the log-probability (0.05 nat), which is the quantity being minimized and has no such failure mode; in the run of record en_city closed in 193 steps without intervention and it_capitale ran to the cap. What the old criterion did to these two facts in development is in Section 5.

5 Development observations (not yet reproducible from this repository)

The observations in this section were made during development with diagnostic tools that are not yet part of the public repository, on the same model and facts. We report them because they explain the results above and shape the next steps, and we flag them so that they are not read as verified: the numbers here cannot be regenerated from the repository today, and will be added with their tools in a later version.

The false plateau. Under the p_{free} criterion, en_city and it_capitale both stopped at step 151; continued with the log-probability criterion from their stopped rows, en_city was still descending and closed 76 steps later, it_capitale ran to the cap. That observation is what made the log-probability criterion the default.

The cost of a graft is set by the base model's prior at the trigger. One forward of the base model at each trigger gives the probability of its top-1 next token. Ordering the eight facts by that probability almost reproduces the order of descent steps (Spearman $\rho = 0.83$): the three slow facts had top-1 probability 0.34 (it_capitale), 0.28 (en_city) and 0.19 (en_planet); the five fast ones 0.04–0.12. The rank or log-probability of the target itself does not matter (it_mestiere: rank 6973, the fastest); the magnitude of the gradient on the rows is *larger* for the slow facts, not smaller. The competitor is not the true fact: after *La capitale della Francia è* the base model's top-1 is the article *una* (0.34), and *Parigi* is not in the top five; after *Oliver Hale was born in* it is a space token, then *Boston* (0.25). A name can displace a name; the eight rows at block 1 could not displace a syntactic form. We read this as a triage rule: top-1 probability, entropy and the identity of the top-1 at the trigger predict, before any descent, whether a trigger should be reformulated.

Every regression is a routing change. In all descents, every step in which the log-probability decreased coincided with a change of expert routing in at least one layer (314 of 315 regressions across the eight facts, the single exception at step 289 of en_planet), and the changes concentrate in blocks 16–47. Refresh is what makes the descent see them.

Frozen-routing probes. Descending with the routing frozen at step 0 produces no regressions and a smooth curve, but the probability it reports is not real: measured afterwards under free routing, the frozen-routing checkpoints gave $p_{\text{free}} = 0.23$ (it_mestiere, versus 0.95 reported), 0.004 (it_capitale) and 1.8×10^{-5} (en_city). For a fact the model is undecided about (it_mestiere), the descent closes with or without refresh; for an intermediate case (en_city) refresh is the mechanism by which the model changes its mind, and the frozen descent does not close in 300 steps; for a decided case (it_capitale) nothing closes. Refreshing the routing every 5 steps instead of every step, halving the learning rate, or changing the random seed did not change it_capitale's outcome (these four variants inherited the old p_{free} plateau criterion and stopped at 151 steps, against the 300 of the baseline).

6 Related work

DeepSeek's Engram [1] introduced the conditional-memory table this method edits, and released models built on it. Qwen3.8-Flash-Next carries the same design; its llama.cpp support [6] is the base of the engine fork used here. *User as Engram* [2] stores per-user memory as local edits of a hash-keyed table and is the closest work: it writes a row in one step through the unembedding projection, optionally refined by a few gradient steps through the model, and optimizes many facts jointly, on small Engram models of its own (178M–1.22B parameters, a 50K-row table) whose lookup is read at a late layer. ENGRAFT differs in the target (a 125B mixture-of-experts model in production, its own table at block 1), in refreshing the expert routing at each step, in verifying every number on the real inference engine, and in reporting the document-context failure. One of its findings is worth setting against ours: it reports that writing a fact into a lookup read at an early layer drops recall from near-perfect to about a quarter. Our seven grafts at block 1 reach $p = 0.85$–0.96 on the engine. Models, tables and gates differ, so this is a contrast to investigate, not a contradiction; but it means the early position of the table is not by itself an obstacle to writing into it. *Engram Adapter* [3] and *Memory Grafting* [4] use conditional memory for domain specialization and pre-training, with training rather than post-hoc edits. *ngram-knowledge-injector* [5] patches the same model's table with overlay files by a different construction of the rows. Locate-and-edit methods for dense transformers [7, 8] rewrite MLP weights in closed form; the analogue here is an explicit memory with an exact key. ENGRAFT is not related to ENGRAFT the Byzantine consensus protocol (CCS 2022) or to the Engraft live-programming API (UIST 2023).

7 Limitations and next steps

One model, one engine fork, one run, eight facts. Nothing here measures capacity or interference at hundreds of facts; the eight overlays are disjoint by construction and the zero-interference result says only that. The corpus drift check is exact but weak, since the reference texts never read a grafted row; a text containing the triggers is the right test. The grafts do not generalize to the fact stated in a paragraph, nor to paraphrases that read the same rows. Counterfactual facts are stress tests, not use cases. The replica needs 67 GB of RAM for this model.

The next steps follow from Sections 4.1 and 5: descend on a loss summed over several contexts that read the same rows (bare trigger, the sentence in the document, same-tail paraphrases); triage triggers by the base model's prior before grafting and reformulate the decided ones; measure drift on texts that contain the triggers; and port the diagnostic tools so that Section 5 becomes reproducible. We would like the method checked on DeepSeek's own Engram models.

Reproducibility

`scripts/reproduce.sh <date>` runs the three steps end to end and needs the model GGUF, its tokenizer, the table GGUF and a build of the engine fork at the recorded commit; the script refuses a build at a different commit. Without a model, the same code path runs against fakes in about a minute (`README.md`, Quick start). The repository is released under Apache 2.0; this report under CC BY 4.0; overlays built against Qwen3.8-Flash-Next fall under the Qwen Community License.

References

[1] X. Cheng et al. Conditional Memory via Scalable Lookup: A New Axis of Sparsity for Large Language Models. arXiv:2601.07372, 2026. `https://github.com/deepseek-ai/Engram`.

[2] B. Li. User as Engram: Internalizing Per-User Memory as Local Parametric Edits. arXiv:2606.19172, 2026.

[3] J. Hou et al. When to Adapt: Conditional Memory Adapters for Retention-Preserving Domain Specialization. arXiv:2608.29327, 2026.

[4] R. Cheng et al. Memory Grafting: Scaling Language Model Pre-training via Offline Conditional Memory. arXiv:2605.20948, 2026.

[5] A. Ortega. ngram-knowledge-injector. `https://github.com/ortegaalfredo/ngram-knowledge-injector`, 2026.

[6] D. Han. model: add Qwen3.8-Flash-Next (qwen4exp). ggml-org/llama.cpp pull request 27742, 2026.

[7] K. Meng, D. Bau, A. Andonian, Y. Belinkov. Locating and Editing Factual Associations in GPT. NeurIPS 2022.

[8] K. Meng, A. Sen Sharma, A. Andonian, Y. Belinkov, D. Bau. Mass-Editing Memory in a Transformer. ICLR 2023.